Experiment No.07

```
#include<stdio.h>
#include<stdlib.h>
#include<conio.h>
#include<time.h>
#include<math.h>
#define   size_x  384
#define   size_y  288
typedef struct{
            int imagesize_x, imagesize_y;    int **pixel; }

image_t;  image_t allocate_image(const int imagesize_x, const int imagesize_y);  int mod(int z, int l);

int main()  {
  image_t   image_in,image_out;
  int m,n, temp;    int smooth,csx,csy;    int k,l,x,y,sum1,sum2;   FILE *file_in,*file_out;  char
dummy[50]="";

//Laplacian Operator
 static int w1[3][3]={   {0,-1,0},
                        {-1,4,-1},
                         {0,-1,0}  };

 static int w2[3][3]={   {0,-1,0},
                        {-1,4,-1},
                         {0,-1,0}   };

  file_in=fopen("i7.pgm", "r+");    file_out=fopen("edge_laplacian.pgm", "w+");
  fgets(dummy,50,file_in);
  do  {  fgets(dummy,50,file_in); }  while(dummy[0]=='#');  fgets(dummy,50,file_in);
  fprintf(file_out,"P2\n%d %d\n255\n",(size_x),(size_y));

  image_in =  allocate_image(size_x,size_y);  image_out = allocate_image(size_x,size_y);
  //Reading Input Image
  for (n = 0; n < size_y; n++)  {
    for (m = 0; m <size_x; m++)     {
                    fscanf(file_in,"%d",&temp);    image_in.pixel[m][n] = temp;     }  }

  /* Edge Detection  */
 smooth=3;  csx=smooth/2;  csy=smooth/2;

 //Edge detection
 for (n = 0; n < size_y; n++)  {
```

```
  for (m = 0; m < size_x; m++)  {
    sum1=0;sum2=0;
    for(k=0;k<smooth;k++)   {
      for(l=0;l<smooth;l++)        {
                x=m+k-csx; y=n+l-csy;
                sum1+=w1[k][l]* image_in.pixel[mod(x,size_x)][mod(y,size_y)];
         sum2+=w2[k][l]* image_in.pixel[mod(x,size_x)][mod(y,size_y)];
      }     }
    if((fabs(sum1)+fabs(sum2))>125)       image_out.pixel[m][n]=255;
    else       image_out.pixel[m][n]=0;
   }   }

  //Writing Edge Detected Image
  for (n = 0; n < size_y; n++)  {
   for (m = 0; m <size_x; m++)     {
                    fprintf(file_out,"%d ",image_out.pixel[m][n]);
    }  }  }

image_t allocate_image(const int imagesize_x, const int imagesize_y) {
  image_t result;
  int x =  0, y = 0;

  result.imagesize_x = imagesize_x;
  result.imagesize_y = imagesize_y;

  result.pixel =(int **) calloc(imagesize_x, sizeof(int*));

  for(x = 0; x < imagesize_x; x++)  {
    result.pixel[x] =(int*) calloc(imagesize_y, sizeof(int));
    for(y = 0; y < imagesize_y; y++)    {
      result.pixel[x][y] = 0;    }
  }
  return result;
}


 int mod(int z, int l) {
    if( z >= 0 && z < l )  return z;
    else if( z < 0)         return (z+l);
    else if( z > (l-1))   return (z-l);
 }
```

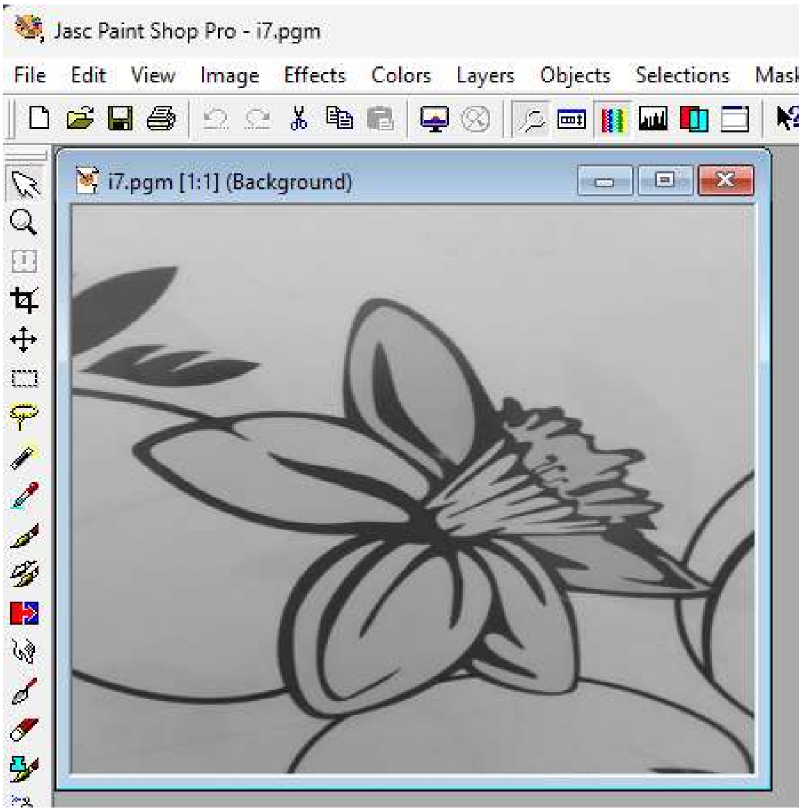

Figure 7(a): Original grayscale image (i7.pgm)

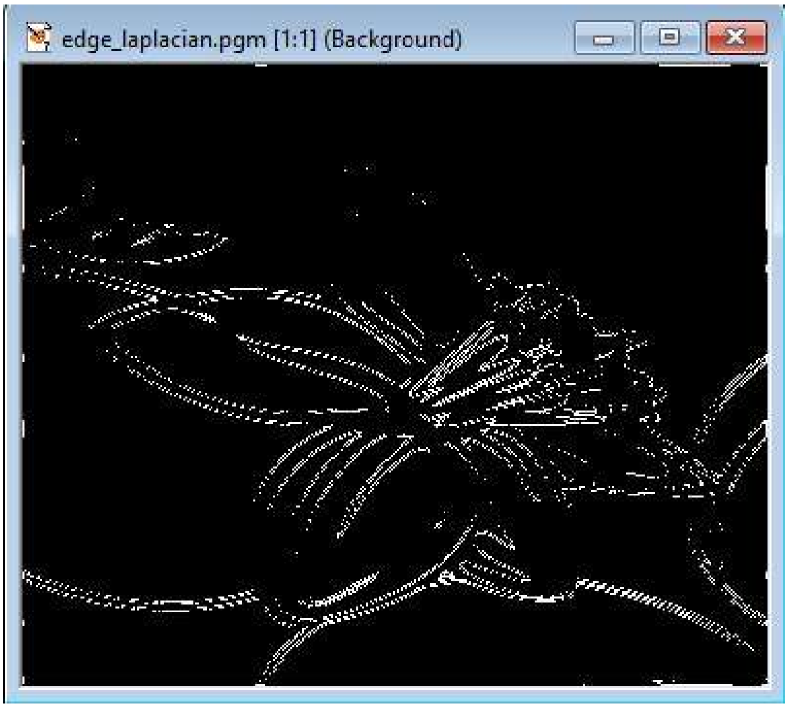

Figure 7(b): Edge-Laplacian image (edge_laplacian.pgm)